

Android API Integration

Cheat Sheet

Transcil Mobile App

Learning reference: Retrofit, OkHttp, DTO, Repository, ViewModel

Purpose: Reading and revision notes

Scope: GET API learning implemented in the current project

Date: July 2026

One-line takeaway

UI asks ViewModel -> ViewModel asks Repository -> Repository uses Retrofit -> JSON becomes a DTO -> UI shows the result.

1. The full call chain

Every network call in a clean Android app should follow this order. Activities and Fragments must not call Retrofit directly.

```
UI (Activity / Fragment)
  -> ViewModel
    -> Repository
      -> Api interface (Retrofit)
        -> ApiClient (OkHttp + Retrofit)
          -> Internet
            -> JSON response
              -> DTO (Kotlin data class)
```

2. Concepts we implemented

INTERNET permission

What: A declaration in AndroidManifest.xml.

Why: Android blocks network until permission is granted. Without it, calls fail.

Retrofit

What: Library that turns a Kotlin interface into HTTP calls.

Why: You declare @GET/@POST methods instead of writing raw HTTP code.

OkHttp

What: HTTP engine that opens the real network connection.

Why: Retrofit uses OkHttp under the hood. Configured once in ApiClient.

Gson converter

What: Converts JSON text to Kotlin objects and back.

Why: Servers send text; your app needs typed fields like `post.title`.

Coroutines (suspend + `viewModelScope.launch`)

What: Run network work off the main (UI) thread.

Why: Network on the main thread freezes or crashes the app.

BASE_URL (BuildConfig)

What: Root server address stored in build config.

Why: One place to switch environments. Must end with a trailing slash (/).

DTO (Data Transfer Object)

What: A Kotlin data class shaped like the JSON response.

Why: Maps server JSON into typed fields your app can use safely.

Api interface (DemoApi)

What: Interface that lists endpoints as functions.

Why: One method = one HTTP call. Clear, testable, reusable.

ApiClient

What: Object that builds Retrofit and OkHttpClient once.

Why: Avoid recreating the client on every screen.

Repository (DemoRepository)

What: Middle layer between ViewModel and Api.

Why: Hides networking from UI. Easy to swap stub data for real APIs later.

ViewModel (DemoViewModel)

What: Decides when to call the API; exposes loading, success, and error.

Why: Keeps UI thin. Reuses BaseViewModel loading/error helpers.

Activity / UI

What: Triggers the call and observes LiveData results.

Why: UI should not build URLs or parse JSON. Only show results.

Logging interceptor

What: OkHttpClient interceptor that prints request/response in Logcat.

Why: Helps debug what was sent and received while learning.

3. Not implemented yet (planned)

Auth interceptor

What: Adds Authorization: Bearer <token> on every request.

Why: Protected APIs need a login/OTP token.

Needs: A real token from OTP/login first.

Authenticator

What: On HTTP 401, refresh the token and retry the call.

Why: Handle expired tokens automatically.

Needs: Refresh-token API from backend.

4. GET vs POST

Type	Use	Our project
GET	Read data (usually no body)	Practice: GET posts/1
POST	Send data / perform action	OTP send/verify (later)

5. Key code patterns

DTO example

```
data class PostDto(  
    val userId: Int,  
    val id: Int,  
    val title: String,  
    val body: String  
)
```

Rule: property names must match JSON keys (or use @SerializedName).

Api interface example

```
interface DemoApi {  
    @GET("posts/{id}")  
    suspend fun getPost(@Path("id") id: Int): PostDto  
}
```

At runtime this becomes: BASE_URL + posts/1

Repository example

```
class DemoRepository {  
    suspend fun loadPost(id: Int): PostDto {  
        return ApiClient.demoApi.getPost(id)  
    }  
}
```

ViewModel example

```
fun loadPracticePost() {  
    viewModelScope.launch {  
        showLoading()  
        try {  
            val post = repository.loadPost(1)  
            _postTitle.value = post.title  
        } catch (e: Exception) {  
            showError(e.message ?: "Network error")  
        } finally {  
            hideLoading()  
        }  
    }  
}
```

6. Where and why we used it in this project

This section maps each learning piece to the TranscilMobileApp codebase and explains why it was added there.

INTERNET permission

Where: `app/src/main/AndroidManifest.xml`

Why in this project: Required before any Retrofit call can reach the network on a device/emulator.

Library versions

Where: `gradle/libs.versions.toml`

Why in this project: Central catalog for Retrofit, OkHttp, Gson converter, and Coroutines versions.

App dependencies + BASE_URL

Where: `app/build.gradle.kts`

Why in this project: Wires libraries into the app module. `BuildConfig.BASE_URL` currently points to `https://jsonplaceholder.typicode.com/` for safe practice.

PostDto

Where: `app/.../data/model/PostDto.kt`

Why in this project: Matches jsonplaceholder /posts JSON so Gson can parse title and body.

DemoApi

Where: `app/.../data/network/DemoApi.kt`

Why in this project: Declares the practice GET `posts/{id}` endpoint used for first successful call.

ApiClient

Where: `app/.../data/network/ApiClient.kt`

Why in this project: Single shared Retrofit/OkHttp setup. Includes logging interceptor for Logcat debugging.

DemoRepository

Where: `app/.../data/repository/DemoRepository.kt`

Why in this project: Hides ApiClient from ViewModel. Matches the Repository style used by home/KYC stubs.

DemoViewModel

Where: `app/.../data/demo/DemoViewModel.kt`

Why in this project: Practice coroutine call with loading/error via BaseViewModel helpers.

Temporary UI proof (removed after success)

Where: `auth/WelcomeActivity.kt` (cleaned)

Why in this project: Long-press on Welcome title triggered GET and showed Toast 'API OK: ...'. Removed afterward so Welcome stays OTP-only for product flow.

Existing pattern reused: BaseViewModel

Where: `core/BaseViewModel.kt`

Why in this project: Already exposed `isLoading` and `errorMessage`. Network learning reuses those helpers instead of inventing a new error channel.

Existing pattern reused: stub repositories

Where: `home/*Repository.kt`, `kyc/KycProgressRepository.kt`

Why in this project: Home and KYC already use Repository boundaries with local/stub data. Same boundary will later host real API calls without rewriting screens.

Auth screens (not network-wired yet)

Where: `WelcomeViewModel.kt`, `VerifyOtpViewModel.kt`

Why in this project: Still local validation only (10-digit mobile, 6-digit OTP). Next backend step after GET practice: POST send/verify OTP using this same stack.

7. Project file map

File	Role in project
<code>AndroidManifest.xml</code>	INTERNET permission
<code>libs.versions.toml</code>	Dependency versions
<code>app/build.gradle.kts</code>	Deps + <code>BuildConfig.BASE_URL</code>
<code>data/model/PostDto.kt</code>	Practice response DTO
<code>data/network/DemoApi.kt</code>	Practice Api interface
<code>data/network/ApiClient.kt</code>	Retrofit + OkHttp client
<code>data/repository/DemoRepository.kt</code>	Practice repository
<code>data/demo/DemoViewModel.kt</code>	Practice ViewModel
<code>core/BaseViewModel.kt</code>	Shared loading/error helpers
<code>auth/WelcomeActivity.kt</code>	UI (practice hook removed)

8. One-line glossary

DTO: Kotlin class shaped like the JSON

Api interface: List of endpoints Retrofit implements

ApiClient: Shared Retrofit/OkHttp factory

Repository: Place that actually calls the Api

ViewModel: Starts call; manages loading/error/success

Interceptor: Runs on every request (log or add headers)

Authenticator: Handles 401 and token refresh

suspend: Function designed to run inside a coroutine

LiveData: UI observes results from ViewModel

BASE_URL: Server root used by all Retrofit paths

9. What we proved and what is next

Proved

- Practice GET to `https://jsonplaceholder.typicode.com/posts/1` succeeded.
- Toast showed a real server title (API OK: ...).
- OkHttp logging can show request/response bodies in Logcat.
- Welcome screen was cleaned; network learning files were kept for next steps.

Next

- Get a real Transcil GET endpoint (URL + sample JSON) from seniors.
- Wire that GET using the same DTO -> Api -> Repository -> ViewModel path.
- After OTP/token exists: add AuthInterceptor, then Authenticator.

End of cheat sheet. When backend contracts change, update BuildConfig.BASE_URL and DTO field names to match the API documentation.